

SageBridge

Créer des factures dans Sage 50 Gestion commerciale
par l'interface COM — documentation technique et guide

Version 1.2.2 — 20 octobre 2026

github.com/Gnaris/SageBridge

Sommaire

1. Introduction . . . 3
Ce que fait SageBridge . . . 3
Lecture rapide . . . 3
2. Pourquoi l'interface COM . . . 4
3. Architecture . . . 4
Déroulement d'une facture . . . 5
4. Installation, mises à jour, désinstallation . . . 5
Prérequis . . . 5
Installer . . . 5
Premier lancement . . . 6
Mises à jour automatiques . . . 6
Publier une nouvelle version . . . 6
Désinstaller . . . 6
5. Référence de l'interface COM Sage 50 . . . 7
5.1 Inscription et bibliothèques . . . 7
5.2 Connexion et ouverture du dossier . . . 7
5.3 Les moteurs (CreateEngine) . . . 7
5.4 Créer une facture . . . 8
5.5 Lire une table (ITable) . . . 8
5.6 Créer une fiche client . . . 9
5.7 Messages de Sage (IVerif) . . . 9
6. Règles de SageBridge . . . 10
6.1 Recherche du client . . . 10
6.2 Création du client . . . 10
6.3 Régime de TVA . . . 10
6.4 SIRET et N° TVA . . . 10
6.5 Doublons, référence, descriptions . . . 11
6.6 Sage doit être ouvert . . . 11
7. API HTTP locale . . . 11
Requête POST /facture . . . 11
Réponse . . . 11
Sécurité . . . 12
8. Erreurs rencontrées et corrigées . . . 12
9. Limites connues . . . 14
10. Dépannage . . . 14
11. Fichiers et emplacements . . . 15
Code source (github.com/Gnaris/SageBridge) . . . 15

1. Introduction

SageBridge crée des factures directement dans **Sage 50 Gestion commerciale** (anciennement Ciel Gestion commerciale), sans fichier d'import et sans ressaisie. Il reçoit une commande (lignes, prix, adresse du client), retrouve ou crée la fiche client dans le dossier Sage, applique le bon régime de TVA, puis enregistre une **facture provisoire** : elle reste modifiable et supprimable dans Sage jusqu'à sa validation.

Le programme a été conçu pour une boutique de bijoux qui vend en B2B sur plusieurs places de marché (Faire, Ankorstore, Paris Fashion Shops, etc.). L'extension Chrome « Regroupement facture » regroupe déjà chaque commande par catégorie et par prix : SageBridge transforme ce regroupement en facture Sage en un clic. Il reste générique : n'importe quel outil capable d'envoyer du JSON sur le PC peut l'utiliser.

Ce document sert à la fois de **guide d'utilisation** et de **référence technique**. Sage ne publie aucune documentation de son interface COM : tout ce qui figure au chapitre 5 a été établi en lisant les bibliothèques de types installées avec Sage (`WGC.tlb`, `CamCOM.tlb`) et en testant chaque appel sur un vrai dossier (Sage 50 version 6.71).

Ce que fait SageBridge

- **Facture provisoire** avec articles, quantités, prix unitaires HT, taux de TVA et descriptions multi-lignes (couleurs, références).
- **Client automatique** : recherche par nom + adresse + code postal + ville + pays, sans tenir compte des accents ni des majuscules ; création de la fiche sinon.
- **TVA selon le pays** : régime Local (France, Monaco), CEE (Union européenne) ou Hors CEE (reste du monde, DOM-TOM compris) ; lignes à 0 % hors France.
- **Anti-doublon** : une même commande n'est pas facturée deux fois sans confirmation.
- **Programme visible** : fenêtre de suivi, historique, journal, icône près de l'horloge ; démarrage avec Windows.
- **Installateur et mises à jour automatiques** depuis GitHub, désinstallation par Windows.

Lecture rapide

Vous voulez…	Chapitre
installer SageBridge et faire une première facture	4. Installation, mises à jour, désinstallation
comprendre pourquoi on passe par le COM	2. Pourquoi l'interface COM
programmer contre Sage 50 vous-même	5. Référence de l'interface COM Sage 50
connaître les règles client / TVA / doublons	6. Règles de SageBridge
appeler SageBridge depuis un autre outil	7. API HTTP locale
comprendre un message d'erreur	8. Erreurs rencontrées et corrigées · 10. Dépannage

2. Pourquoi l'interface COM

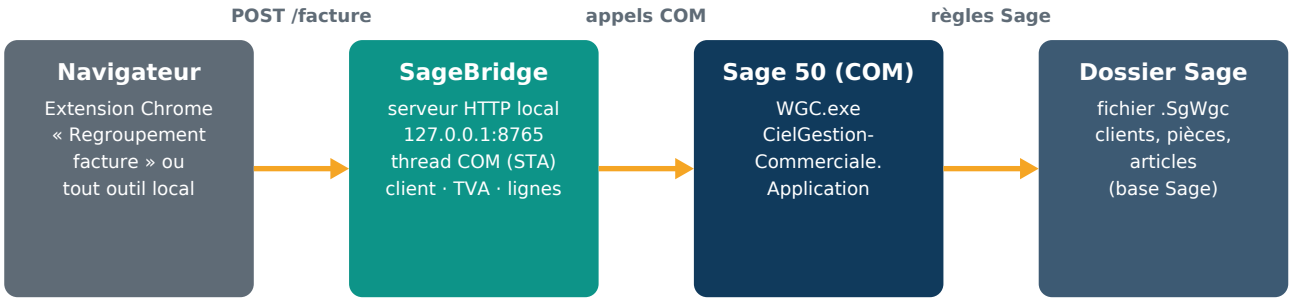
Avant SageBridge, chaque commande était exportée en CSV par l'extension puis importée dans Sage avec un format d'import paramétrable. Cela fonctionne, mais impose plusieurs manipulations par facture, un client générique unique, et une description limitée à une ligne RTF. Quatre façons d'automatiser la création de factures ont été étudiées :

Méthode	Principe	Avantages	Inconvénients	Verdict
Import CSV / Excel	Fichier au format d'import paramétrable, importé à la main	Officiel, sans risque	Manipulations à chaque facture ; client fixe ; pas de création de client ; erreurs peu explicites	Conservé en secours
Saisie simulée	Un robot clique et tape dans Sage	Aucune dépendance technique	Fragile (fenêtres, délais, résolution) ; lent ; impossible à fiabiliser	Écarté
Écriture directe en base	Insérer les lignes dans les fichiers du dossier	Rapide	Format propriétaire non documenté ; contourne les contrôles (numérotation, totaux, stocks) : risque de corrompre le dossier	Écarté
Interface COM	Piloter le moteur de Sage comme le fait Sage lui-même	Numérotation, totaux, TVA et contrôles faits **par Sage** ; messages de Sage récupérables ; pièce provisoire	Non documentée ; Sage doit être installé et ouvert ; autorisation COM à renouveler	**Retenu**

Le COM (*Component Object Model*) est le mécanisme Windows qui permet à un programme d'en piloter un autre. Sage 50 Gestion commerciale expose un objet `CielGestionCommerciale.Application` : on ouvre le dossier, on demande un « moteur » de pièce client, on le remplit, et c'est Sage qui contrôle et enregistre. La facture obtenue est strictement identique à une facture saisie à la main.

> **À retenir**
> Le COM garantit que les règles de Sage s'appliquent (numérotation FA…, calcul des totaux, contrôle des articles et des clients). C'est la seule méthode automatique qui ne risque pas d'abîmer le dossier.

3. Architecture



Tout se passe sur le même PC : rien ne transite par Internet (sauf la vérification des mises à jour sur GitHub).

- **Extension Chrome** : construit la commande (lignes regroupées, codes articles Sage, adresses) et l'envoie depuis son *service worker*. Un content script ne peut pas appeler 127.0.0.1 (protection *Private Network Access* de Chrome) ; le service worker le peut, l'adresse étant déclarée dans `host_permissions`.
- **SageBridge** (C#, .NET Framework 4, un seul fichier source) : serveur HTTP local, file d'attente vers un **thread COM unique en mode STA** (les objets de Sage doivent toujours être appelés depuis le même thread), règles client / TVA, historique et journal.
- **Sage 50** : le serveur COM est `WGC.exe`, le même exécutable que l'application. SageBridge démarre sa propre instance COM, qui travaille sur le dossier déjà ouvert par l'utilisateur.
- **Données de SageBridge** : `%APPDATA%\SageBridge` (réglages, historique anti-doublon, journal). Rien n'est écrit dans le dossier Sage en dehors des appels COM.

Déroulement d'une facture

Étape	Où	Détail
1	Extension	Regroupe la commande, résout les codes articles, construit le JSON (client, livraison, lignes).
2	SageBridge	Contrôle l'origine de la requête, refuse si Sage n'est pas ouvert, vérifie l'anti-doublon.
3	Thread COM	Ouvre le dossier (une fois), lit la table des clients, retrouve ou crée le client (mode TVA selon le pays).
4	Thread COM	Crée la facture provisoire, ajoute les lignes (TVA 0 % si client CEE / Hors CEE), lit numéro et totaux, enregistre.
5	SageBridge	Inscrit la commande dans l'historique, répond `{ok, numero, totaux, client…}` ; l'extension affiche le résultat.

4. Installation, mises à jour, désinstallation

Prérequis

- Windows 10 ou 11 (le .NET Framework 4.8 est fourni avec Windows).
- Sage 50 Gestion commerciale (ex-Ciel) installé sur le même PC. SageBridge détecte Sage grâce à l'inscription de l'objet COM `CielGestionCommerciale.Application`.
- **Autorisation d'accès COM** activée dans Sage : menu **Utilitaires > Mode avancé > Divers > Autorisation accès COM**. Elle est liée au compte Sage et doit être renouvelée environ tous les six mois ; sans elle, l'ouverture du dossier par COM échoue.
- **Sage 50 ouvert** sur le dossier pendant l'utilisation (voir l'erreur 8.10 pour la raison).

Installer

Lancez `SageBridge-Setup.exe`. Le fichier n'étant pas signé numériquement, Windows SmartScreen peut afficher « Windows a protégé votre ordinateur » : *Informations complémentaires* puis *Exécuter quand même*. L'assistant :

- indique si Sage 50 est détecté et si une version de SageBridge est déjà installée (elle sera remplacée, réglages conservés) ;
- propose le dossier d'installation (par défaut `%LOCALAPPDATA%\Programs\SageBridge`), les raccourcis Bureau et menu Démarrer, le démarrage avec Windows ;
- copie `SageBridge.exe`, cette documentation et le désinstalleur, puis inscrit SageBridge dans **Paramètres > Applications** (« Programmes et fonctionnalités ») ;
- affiche la liste des vérifications avant la première facture et lance SageBridge.

Aucun droit administrateur n'est demandé : l'installation est faite pour le compte Windows courant (clés de registre sous `HKEY_CURRENT_USER`). Installation sans questions possible avec `SageBridge-Setup.exe /silent`.

Premier lancement

- SageBridge cherche les dossiers Sage (`*.SgWgc`) sur le PC : un seul → choisi d'office ; plusieurs → liste de choix ; aucun → fenêtre pour l'indiquer.
- **Réglages…** : dossier Sage, client par défaut (utilisé si une commande n'a pas d'adresse exploitable), port local (8765 par défaut), mises à jour automatiques.
- **Facture de test…** : crée une facture provisoire d'une ligne pour un client et un article existants — à supprimer ensuite dans Sage.
- Le bandeau indique en permanence si Sage est ouvert (rafraîchi toutes les 5 secondes) et si le dossier est connecté.

Mises à jour automatiques

Chaque version est publiée comme *release* du dépôt public `github.com/Gnaris/SageBridge`, avec l'installateur et son empreinte SHA-256. SageBridge :

- interroge `api.github.com/repos/Gnaris/SageBridge/releases/latest` **8 secondes après son démarrage puis toutes les heures** (sans compte GitHub) ;
- si la version publiée est plus récente, télécharge `SageBridge-Setup.exe` et `SageBridge-Setup.exe.sha256`, refuse le fichier si l'empreinte ou l'identité du programme ne correspondent pas ;
- attend qu'aucune facture ne soit en cours et qu'aucune fenêtre de SageBridge ne soit ouverte, puis lance l'installateur en mode mise à jour et se ferme ;
- l'installateur remplace les fichiers en conservant les options choisies à l'installation, puis relance SageBridge, qui affiche « SageBridge mis à jour ».
- Si l'installation automatique est désactivée (Réglages) ou si le programme lancé n'est pas la copie installée, une notification signale la nouvelle version ; le bouton **Mises à jour…** permet de vérifier et d'installer à la demande.

Publier une nouvelle version

- Augmenter `AssemblyVersion` / `AssemblyFileVersion` dans `SageBridge.cs` et ajouter une section au `CHANGELOG.md`.
- Lancer `build.cmd` : compile le programme puis l'installateur (qui l'embarque avec la documentation et le logo) et calcule l'empreinte.
- Lancer `publier-version.cmd` : avec le compte GitHub déjà enregistré dans Git sur le PC, pousse le code source, crée la release `vX.Y.Z` (brouillon), y joint l'installateur, l'empreinte et la documentation, puis la publie. Les PC équipés la reçoivent dans l'heure.

Désinstaller

Paramètres > Applications > SageBridge > Désinstaller (ou Panneau de configuration > Programmes et fonctionnalités). Le désinstalleur ferme SageBridge, supprime le programme, les raccourcis, le démarrage automatique et l'inscription ; une case permet de supprimer aussi les données de `%APPDATA%\SageBridge`. Les factures déjà créées dans Sage ne sont jamais touchées.

5. Référence de l'interface COM Sage 50

Établie sur Sage 50 Gestion commerciale 6.71 (dossier en base locale). Les noms sont ceux des bibliothèques de types ; les identifiants numériques ont tous été vérifiés par des appels réels.

5.1 Inscription et bibliothèques

Élément	Valeur
ProgID	`CielGestionCommerciale.Application`
CLSID inscrit	`{10017AE8-0000-84DE-AEB4-0000E80C11B9}`
Serveur (LocalServer32)	`C:\Program Files (x86)\Ciel\Gestion Commerciale\WGC.exe` (serveur COM hors processus)
Bibliothèque « WGestCom »	`WGC.tlb` (dossier d'installation) — moteurs, tables et énumérations de la gestion commerciale
« Ciel Standard Library »	`CamCOM.tlb` (`Common Files\Ciel`) — IApplication, IDocument, ITable, IVerif…
Piège	`WGC.tlb` déclare une coclasse au CLSID …84D8… qui **n'est pas inscrite** : il faut passer par le ProgID.

5.2 Connexion et ouverture du dossier

```
Type t = Type.GetTypeFromProgID("CielGestionCommerciale.Application");
dynamic app = Activator.CreateInstance(t);          // toujours une NOUVELLE instance
string version = app.Name(0);                       // "Sage 50 (6.71)"
dynamic doc = app.OpenFile(@"C:\...\MonDossier.SgWgc", "", "", 0);
string nomDossier = doc.Name;
```

- `Marshal.GetActiveObject` sur le Sage ouvert renvoie un objet qui ne répond à aucune méthode (*E_NOTIMPL*, « méthode non implémentée ») : seule une nouvelle instance fonctionne.
- Les objets COM doivent être créés et appelés depuis **un seul thread STA** ; SageBridge sérialise tous les appels dans une file traitée par ce thread.
- Libérer les objets de travail avec `Marshal.FinalReleaseComObject` (moteurs, tables, vérificateurs). En revanche, **ne pas relâcher l'application ni le dossier** tant que Sage est ouvert : le dossier obtenu par `OpenFile` est celui affiché à l'écran, et sa libération a figé Sage lors des essais (erreur 8.22). SageBridge les garde jusqu'à sa fermeture, puis arrête son processus sans les relâcher.
- Après un redémarrage de Sage, l'ancienne connexion lève *« Le serveur RPC n'est pas disponible » (0x800706BA)* : il faut tout relâcher et rouvrir le dossier.

5.3 Les moteurs (CreateEngine)

`doc.CreateEngine(id)` renvoie un « moteur » : un objet qui saisit une pièce ou une fiche comme le ferait l'écran de Sage, avec les mêmes contrôles.

Identifiant	Moteur	Usage
97001	WgcEngPieceCli	Pièce client : devis, commande, bon de livraison, facture, avoir, acompte
97002	WgcEngLineCli	Ligne de pièce client (à remplir puis ajouter à la pièce)
97004	WgcEngClient	Fiche client

Types de pièce (énumération eWgcType) passés à `Create` : 1 devis, 2 commande, 3 bon de livraison, **4 facture**, 5 avoir, 6 acompte.

5.4 Créer une facture

```
dynamic piece = doc.CreateEngine(97001);
dynamic ligne = doc.CreateEngine(97002);
dynamic verif = app.CreateVerif();
piece.Create(4);                              // facture
piece.Client = "282";                          // code client existant
string nom = piece.Name;                       // vide => client inconnu
piece.reference = "Faire 1234ABCD";            // référence de la pièce (35 car.)
piece.Remarque  = "Commande Faire 1234ABCD";

ligne.RazLine();
ligne.Article = "ART0003";                     // code article existant
ligne.Quantite = 12;
ligne.PUHT = 2.90;                             // prix unitaire HT imposé
ligne.TauxTVA = 20;                            // 0 pour un client CEE / Hors CEE
ligne.Description = "BOUCLES D'OREILLES - A2425\nDoré x6\nArgenté x6";
piece.AddLine(ligne);                          // false => ligne refusée

string numero = piece.Numero;                  // à lire AVANT Save
double ht = piece.TotalHT, ttc = piece.TotalTTC;
bool ok = piece.Save(verif);                   // enregistre la pièce PROVISOIRE
for (int k = 1; k <= verif.Total; k++) Console.WriteLine(verif.Desc(k));
```

- **AddLine** conserve le prix fourni ; `AddLineAutoCalc` recalcule le prix depuis le tarif de l'article (0 € si l'article n'a pas de prix de vente).
- **Description** : texte brut ; les retours à la ligne `\n` sont respectés. Le RTF n'est *pas* interprété par le COM (affiché tel quel), contrairement à l'import CSV.
- **Numéro et totaux** se lisent avant Save : après l'enregistrement le moteur est vidé et toute lecture échoue (« Aucune pièce client n'a été initialisée par le composant COM »).
- La pièce est enregistrée **non validée** : elle apparaît dans la liste des factures, modifiable et supprimable comme une saisie manuelle.

5.5 Lire une table (ITable)

```
dynamic t = doc.OpenTable(97003);          // table des clients
string nomTable = t.TableName;             // "Cli"
int nb = t.NbN;                            // nombre de fiches
for (int i = 1; i <= t.Fields.Count; i++)  // collection numérotée à partir de 1
{ dynamic f = t.Fields.Item(i); Console.WriteLine(f.Id + " " + f.Name + " " + f.Title); }
for (bool ok = t.First(); ok; ok = t.Next())
    Console.WriteLine(t.GetVal(1) + " " + t.GetVal(2));   // Code, Nom
```

Piège des identifiants
Dans `WGC.tlb`, les identifiants de tables et de rubriques sont des valeurs 32 bits du type `0x8C017AEB`. `OpenTable` attend les **bits bas** (`0x017AEB` = 97003) : la valeur complète ouvre une autre table (Région, Marque…) ou renvoie « Paramètres incorrects ». Les rubriques sont numérotées **1, 2, 3…** dans l'ordre de l'énumération ; `Fields.Item(0)` renvoie null.

Table	Id	Table	Id
Clients (Cli)	97003 ✔	Pièces clients (en-têtes)	97004
Articles	97002	Lignes de pièces clients	97006
Représentants	97001	Fournisseurs	97007
Familles clients	97005	Familles articles	97008
Types de pièces clients	97009	Règlements clients	97012
Paramètres	97025	Société	97029
Mouvements de stock	97036	Tarifs	97045

✔ = vérifié par un appel réel ; les autres valeurs sont calculées de la même façon à partir de `WGC.tlb` et se contrôlent avec `GET /diag?id=…` (chapitre 7).

Rubriques utiles de la table des clients (147 rubriques)

Id	Rubrique	Contenu	Id	Rubrique	Contenu
1	Code	code client	25	FacTel	téléphone
2	Nom	nom	32	ModeTVA	Local / CEE / Hors CEE / Suspension de taxe
3	FacSociete	société (facturation)	33	NII	N° TVA intracom
5	FacAdresse1	adresse	46	CpteComptable	compte comptable
6	FacAdresse2	complément	54	Email	e-mail
8	FacCP	code postal	56	SIRET	SIRET
9	FacVille	ville	71	LivTel	téléphone (livraison)
10	FacCtyCode	pays, code ISO 3 (« FRA »)	88	NomCtc	contact (lecture seule par COM)
11	FacCtyName	pays, nom	129	IsProspect	prospect
12–20	Liv…	même bloc pour la livraison	143	TypeCli	Particulier / Professionnel / Administration

SageBridge résout ces rubriques **par leur nom** à l'ouverture du dossier (collection `Fields`) et ne se sert des numéros ci-dessus qu'en secours : une version de Sage qui renumérote ses rubriques reste prise en charge.

5.6 Créer une fiche client

```
dynamic cli = doc.CreateEngine(97004);
cli.Create();
string code = "283";  while (cli.Exist(code)) code = (int.Parse(code) + 1).ToString();
cli.Code = code;
cli.Name = "Bijoux Martin";
cli.SetFieldIdValStr(143, "Professionnel", 0, 0);   // TypeCli : obligatoire
cli.SetFieldIdValStr(32,  "CEE", 0, 0);             // ModeTVA
cli.SetFieldIdValStr(5,   "Hauptstraße 12", 0, 0);  // FacAdresse1
cli.SetFieldIdValStr(10,  "DEU", 0, 0);             // FacCtyCode (ISO 3)
cli.SetFieldIdValStr(11,  "Allemagne", 0, 0);       // FacCtyName
cli.SetFieldIdValStr(15,  " ", 0, 0);               // efface LivAdresse2 pré-remplie
bool ok = cli.Save(verif, ref code);                // code final renvoyé par Sage
```

- `SetFieldIdValStr` renvoie false si Sage refuse la valeur ; le détail arrive dans le vérificateur au Save.
- Sage **pré-remplit la livraison** avec la facturation : pour vider une rubrique de livraison, écrire une espace.
- Le contact (`NomCtc`) n'est pas modifiable par cette méthode (« champ contact refusé ») ; il relève des méthodes de contacts du moteur.
- Un SIRET ou un N° TVA invalide bloque l'enregistrement : les contrôler avant (chapitre 6.4).

5.7 Messages de Sage (IVerif)

`app.CreateVerif()` collecte les messages de contrôle. Après Save : `verif.Total` (nombre), `verif.Desc(k)` pour k = 1…Total, `verif.Reset()` avant un nouvel essai. Exemples réels : « Le type de client doit être renseigné (Particulier, Professionnel ou Administration) », « Le numéro SIRET est incorrect : Numéro SIRET invalide », « Le N° TVA intracom est incorrect : Incohérence avec le numéro SIREN », « Le SIRET doit être renseigné pour un client de type professionnel » (avertissement non bloquant).

6. Règles de SageBridge

6.1 Recherche du client

Chaque texte est **normalisé** avant comparaison : décomposition Unicode (NFD) puis suppression des accents, minuscules, tout caractère non alphanumérique remplacé par une espace, espaces multiples réduites. Le code postal ne garde que lettres et chiffres ; le pays est ramené à son code ISO 2 (noms français, anglais, langue locale, codes ISO 2 et 3 ; pays vide = France).

- Comparés : **nom** (contre `Nom` ou la société du bloc), **adresse 1**, **code postal**, **ville**, **pays** — tous doivent être égaux.
- Le bloc **facturation** de la commande est comparé au bloc facturation *et* au bloc livraison de chaque fiche ; puis le bloc livraison de la commande au bloc livraison des fiches.
- Les prospects sont ignorés. Le premier client qui correspond est utilisé ; son mode TVA est relu dans la fiche.
- Sans adresse exploitable (pas de nom ou pas d'adresse / ville) : client par défaut des Réglages, ou refus explicite s'il n'est pas défini.

6.2 Création du client

- Code : suite du format le plus fréquent du dossier (« 281 » → « 282 », « CLI0041 » → « CLI0042 »), vérifié par `Exist`.
- Type **Professionnel**, société, adresse, complément, CP, ville, pays (ISO 3 + nom français), téléphone, e-mail, SIRET, N° TVA ; bloc livraison renseigné à part.
- Si Sage refuse malgré les contrôles, second essai sans SIRET ni N° TVA plutôt que de bloquer la facture ; les messages sont renvoyés à l'appelant.

6.3 Régime de TVA

Pays de facturation	ModeTVA	Lignes
France métropolitaine, Monaco	Local	taux transmis (20 % en général)
Autres États membres de l'UE : Allemagne, Autriche, Belgique, Bulgarie, Chypre, Croatie, Danemark, Espagne, Estonie, Finlande, Grèce, Hongrie, Irlande, Italie, Lettonie, Lituanie, Luxembourg, Malte, Pays-Bas, Pologne, Portugal, Rép. tchèque, Roumanie, Slovaquie, Slovénie, Suède	CEE	0 %
Reste du monde, dont Suisse, Royaume-Uni, Norvège, États-Unis, DOM (Réunion, Guadeloupe, Martinique, Guyane, Mayotte) et COM	Hors CEE	0 %

Le pays seul décide : un N° TVA absent ne change pas le régime. Pour un client existant, c'est le `ModeTVA` de sa fiche qui s'applique (une fiche corrigée à la main dans Sage est donc respectée). Un pays non reconnu est traité en Hors CEE avec un message à vérifier.

6.4 SIRET et N° TVA

- **SIRET** : 14 chiffres et clé de Luhn ; sinon il n'est pas enregistré (message).
- **N° TVA français** : FR + clé + SIREN, clé = (12 + 3 × (SIREN mod 97)) mod 97, et SIREN identique aux 9 premiers chiffres du SIRET s'il est fourni. Les numéros étrangers sont transmis tels quels.
- Les deux sont facultatifs : beaucoup de commandes n'en portent pas ; Sage n'émet alors qu'un avertissement.

6.5 Doublons, référence, descriptions

- Clé anti-doublon `site|orderNo` enregistrée dans `factures.csv` : un second envoi renvoie HTTP 409 avec le numéro existant ; renvoyer avec `"force": true` pour créer quand même.
- La référence de la pièce reçoit par exemple « Faire 1234ABCD » (place de marché + numéro de commande).
- Descriptions multi-lignes en texte brut (« BOUCLES D'OREILLES - A2425 », puis une ligne par couleur).

6.6 Sage doit être ouvert

Avant chaque appel, SageBridge vérifie qu'un processus `WGC.exe` **avec fenêtre** existe ; sinon il répond immédiatement « Sage 50 n'est pas ouvert ». Chaque opération COM est bornée à 2 minutes. Au démarrage et avant d'ouvrir Sage, il ferme les instances `WGC.exe` **sans fenêtre** lancées depuis plus d'une minute alors qu'aucun Sage visible n'est ouvert (COM orphelin) — jamais un Sage ouvert par l'utilisateur.

7. API HTTP locale

SageBridge écoute sur `http://127.0.0.1:8765/` et `http://localhost:8765/` uniquement : il n'est pas joignable depuis un autre PC.

Méthode	Adresse	Réponse
GET	/ping	{ ok, app, version, sageOuvert, sageFenetre, dossier, clientParDefaut }
POST	/facture	crée la facture (ou la simule) — voir ci-dessous
GET	/clients	[{ code, nom, ville, pays, modeTVA, tva, prospect }]
GET	/diag?id=97003&fids=1,2,3	outil de développement : nom de la table, rubriques, valeurs de la première fiche

Requête POST /facture

```
{
  "site": "faire", "orderNo": "1234ABCD",
  "reference": "Faire 1234ABCD", "remarque": "",
  "client": "",                 // code client imposé si aucune adresse (sinon réglage)
  "force": false,               // true : ignorer l'anti-doublon
  "simulation": false,          // true : tout vérifier sans rien créer
  "customer": { "company": "Bijoux Martin", "contact": "Claire Martin",
                "address1": "12 rue des Lilas", "address2": "", "postalCode": "69003",
                "city": "Lyon", "country": "France", "phone": "", "email": "",
                "siret": "", "vat": "" },          // bloc facturation
  "shipping": { ... même forme ... },             // bloc livraison (facultatif)
  "lines": [ { "code": "ART0003", "qty": 12, "puht": 2.9, "tva": 20,
               "description": "BOUCLES D'OREILLES - A2425\nDoré x6\nArgenté x6" } ]
}
```

Réponse

```
HTTP 200  { "ok": true, "numero": "FA10005788", "totalHT": 34.8, "totalTTC": 41.76,
            "nbLines": 1, "clientCode": "282", "clientName": "Bijoux Martin",
            "clientCreated": true, "clientMatched": false, "clientModeTVA": "Local",
            "simulation": false, "messages": [ "Client : Le SIRET doit être renseigné…" ] }
HTTP 409  { "ok": false, "duplicate": true, "numero": "FA10005788", "error": "Commande déjà envoyée…" }
HTTP 500  { "ok": false, "error": "Ligne 1 (article « ART9999 ») refusée par Sage…" }
```

Sécurité

- Les requêtes portant un en-tête `Origin` ne sont acceptées que d'une **extension de navigateur** (chrome-extension://, moz-extension://) ou de SageBridge lui-même ; les outils locaux (PowerShell, scripts) n'envoient pas d'origine et sont acceptés ; une requête marquée `Sec-Fetch-Site: cross-site` est refusée (HTTP 403).
- Les en-têtes CORS ne sont renvoyés qu'à une origine autorisée : une page web ouverte dans le navigateur ne peut ni créer de facture, ni lire la liste des clients.
- Corps limité à 2 Mo, réponses `nosniff` et `no-store`.
- Mises à jour : uniquement depuis github.com en HTTPS, empreinte SHA-256 publiée avec l'installateur et nom de produit contrôlés avant exécution.

8. Erreurs rencontrées et corrigées

Chaque problème ci-dessous a été rencontré pendant le développement, sur un vrai dossier. La cause et la correction sont décrites pour qu'il ne se reproduise pas.

N°	Symptôme	Cause	Correction
8.1	**Classe non enregistrée (REGDB_E_CLASSNOTREG)**	La coclasse déclarée dans WGC.tlb (CLSID …84D8…) n'est pas inscrite dans le registre.	Lire le registre : le ProgID `CielGestionCommerciale.Application` pointe vers le CLSID …84DE… servi par WGC.exe. SageBridge passe toujours par le ProgID.
8.2	**« Méthode non implémentée » sur OpenFile**	L'objet obtenu par GetActiveObject (instance déjà ouverte) n'expose pas l'automation.	Créer une nouvelle instance avec Activator.CreateInstance ; elle travaille sur le même dossier que le Sage ouvert.
8.3	**« Aucune pièce client n'a été initialisée par le composant COM »**	Lecture du numéro et des totaux après Save : le moteur est remis à zéro par l'enregistrement.	Lire Numero, TotalHT et TotalTTC avant Save.
8.4	**Prix à 0 € sur les lignes**	AddLineAutoCalc applique le tarif de l'article, vide dans le dossier.	AddLine avec PUHT explicite (le prix vient de la commande).
8.5	**Description affichée « {\rtf1\ansi… »**	Le COM stocke la description telle quelle, sans interpréter le RTF (seul l'import CSV l'interprète).	Envoyer du texte brut avec retours à la ligne ; SageBridge convertit tout RTF reçu en texte.
8.6	**Mauvaise table ouverte, « Paramètres incorrects »**	Les identifiants de WGC.tlb sont des valeurs 32 bits ; OpenTable attend les bits bas.	Masquer la valeur (0x8C017AEB → 0x017AEB = 97003) et numéroter les rubriques 1..N ; contrôle du nom de table (« Cli ») et rubriques résolues par nom.
8.7	**Fields.Item(0) renvoie null**	La collection des rubriques est numérotée à partir de 1.	Parcourir de 1 à Count.
8.8	**Aucun client lu, programme bloqué**	Un dictionnaire de pays initialisé avec une clé en double (« usa ») levait une exception à l'initialisation de la classe, rendant toute normalisation impossible.	Table de pays construite par affectation (un doublon ne peut plus planter) et testée.
8.9	**Client existant non reconnu (pays)**	Sage stocke le pays en code ISO 3 (« FRA », « DEU ») alors que les commandes portent un nom ou un code ISO 2.	Clé pays commune ISO 2 calculée depuis noms FR/EN/locaux, ISO 2 et ISO 3 ; création avec code ISO 3 + nom français.

N°	Symptôme	Cause	Correction
8.10	**Requête bloquée plus de 45 s, Sage ne s'ouvre plus**	Sage fermé : l'activation COM démarre un WGC.exe sans fenêtre qui attend indéfiniment et verrouille le dossier.	Refus immédiat tant qu'aucun Sage avec fenêtre n'est ouvert, délai maximal de 2 minutes, fermeture des instances sans fenêtre orphelines.
8.11	**« Le type de client doit être renseigné »**	TypeCli est obligatoire à la création.	TypeCli = « Professionnel » (activité B2B).
8.12	**« SIRET incorrect » / « N° TVA… Incohérence avec le numéro SIREN »**	Identifiants fournis par la place de marché parfois faux ou incohérents ; Sage refuse alors toute la fiche.	Contrôle Luhn et clé TVA avant envoi ; second enregistrement sans SIRET ni TVA si Sage refuse encore.
8.13	**« champ contact refusé »**	NomCtc n'est pas modifiable par SetFieldIdValStr.	Contact non écrit (la fiche reste valide).
8.14	**Adresse de livraison égale à la facturation**	Sage recopie la facturation dans la livraison à la création.	Écrire explicitement chaque rubrique de livraison ; une espace pour vider.
8.15	**« Le serveur RPC n'est pas disponible » (0x800706BA)**	Sage a été fermé ou redémarré : la connexion COM est morte.	Relâcher les objets, se reconnecter et rejouer l'opération une fois.
8.16	**Appel refusé par Chrome depuis la page**	Private Network Access : une page publique ne peut pas appeler 127.0.0.1.	Appel depuis le service worker de l'extension, adresse déclarée dans host_permissions.
8.17	**Compilation impossible : SageBridge.exe verrouillé**	Le programme en cours d'exécution verrouille son fichier.	build.cmd / l'installateur ferment SageBridge avant de remplacer le fichier.
8.18	**Toute page web pouvait créer des factures (1.1)**	CORS « * » et autorisation réseau privé envoyés à toutes les origines.	1.2 : origines limitées aux extensions et aux outils locaux, refus des requêtes inter-sites.
8.19	**Neuf pays de l'UE classés Hors CEE (1.1)**	Bulgarie, Croatie, Chypre, Estonie, Lettonie, Lituanie, Malte, Slovaquie, Slovénie absents de la table.	1.2 : table complète des 27 États membres et de leurs noms, « EL » pour la Grèce, DOM-TOM en Hors CEE.
8.20	**Sage ouvert fermé de force (1.1)**	Une instance avec fenêtre jugée « figée » après 4 s était tuée, au risque de perdre une saisie en cours.	1.2 : seules les instances sans fenêtre de plus d'une minute, sans Sage visible ouvert, sont fermées.
8.21	**Réglages propres à un seul dossier (1.1)**	Nom du dossier, client 003 et article ART0001 écrits dans le code.	1.2 : dossier choisi au premier lancement parmi ceux du PC, client par défaut et article de test réglables.
8.22	**Sage 50 figé après une mise à jour de SageBridge (1.2.0 et 1.2.1)**	En se fermant, SageBridge relâchait (FinalReleaseComObject) l'application et le dossier COM ; le dossier étant celui ouvert à l'écran, Sage s'est bloqué et la fermeture de SageBridge aussi (arrêt forcé par l'installateur au bout de 30 s).	1.2.2 : aucun appel COM à la fermeture, arrêt immédiat du processus (comme un taskkill, sans effet sur Sage). Si Sage est figé : Gestionnaire des tâches, fin de tâche, puis le rouvrir.
8.23	**Sage non détecté par l'installateur**	WGC.exe est un serveur COM 32 bits : son CLSID est inscrit dans la vue 32 bits du registre, invisible depuis un programme 64 bits.	Lecture du registre dans les deux vues (RegistryView.Registry32 puis Registry64), repli sur le chemin d'installation standard.
8.24	**Publication GitHub interrompue (HTTP 408)**	Envois longs coupés par la connexion (git push et envoi de l'installateur).	Envoi du code par l'API fichier par fichier, nouvel essai automatique des envois, reprise du brouillon de version, Expect: 100-continue désactivé.

9. Limites connues

- Sage 50 doit être installé sur le même PC et **ouvert** sur le dossier ; un seul dossier à la fois.
- L'autorisation d'accès COM expire (environ 6 mois) : la renouveler dans Sage quand l'ouverture du dossier échoue.
- Les articles doivent exister dans Sage ; une ligne avec un code inconnu fait échouer la facture (rien n'est créé).
- Le contact de la fiche client n'est pas renseigné ; les clients sont créés en type Professionnel.
- Les factures sont créées **provisoires** : la validation reste un geste humain dans Sage.
- Le régime de TVA est déduit du pays ; les cas particuliers (franchise, suspension de taxe, autoliquidation spécifique) se règlent sur la fiche client dans Sage.
- L'installateur n'est pas signé : avertissement SmartScreen au premier lancement d'un fichier téléchargé par le navigateur (les mises à jour automatiques n'en déclenchent pas).

10. Dépannage

Message ou situation	Que faire
« SageBridge ne répond pas » (extension)	Lancer SageBridge (menu Démarrer ou Bureau) ; vérifier que la fenêtre indique « En attente de factures » et le port 8765.
« Sage 50 n'est pas ouvert sur ce PC »	Ouvrir Sage 50 sur le dossier, attendre l'indicateur vert « Sage 50 ouvert », renvoyer.
« Sage a refusé d'ouvrir le dossier »	Renouveler l'autorisation COM (Utilitaires > Mode avancé > Divers > Autorisation accès COM) ; vérifier le dossier dans Réglages.
Sage ne s'ouvre plus / reste invisible	Fermer SageBridge, puis le relancer : il ferme les instances de Sage invisibles ; ou Gestionnaire des tâches > WGC.exe sans fenêtre.
« Ligne n (article …) refusée »	Le code article n'existe pas dans le dossier Sage : le créer ou corriger le code.
« Commande déjà envoyée : facture FA… »	Normal : vérifier la facture existante ; confirmer dans l'extension pour en créer une seconde.
« Impossible d'écouter sur le port 8765 »	Un autre programme utilise le port (ou SageBridge est lancé deux fois) : changer le port dans Réglages et dans l'extension.
Mauvais client choisi	Les cinq informations (nom, adresse, CP, ville, pays) étaient identiques à une fiche existante : corriger la facture provisoire dans Sage.
« Vérification des mises à jour impossible »	Pas d'accès Internet ou GitHub indisponible : sans conséquence, nouvel essai dans l'heure.
Avertissement SmartScreen à l'installation	Informations complémentaires > Exécuter quand même (installateur non signé).

Le **journal** (bouton Journal) détaille chaque opération et chaque message de Sage ; l'installation a son propre journal `installation.log`.

11. Fichiers et emplacements

Emplacement	Contenu
%LOCALAPPDATA%\Programs\SageBridge\	SageBridge.exe, SageBridge-Uninstall.exe, SageBridge - Documentation.pdf
%APPDATA%\SageBridge\settings.ini	dossier, port, client par défaut, article de test, mises à jour automatiques, dépôt
%APPDATA%\SageBridge\factures.csv	historique des factures créées (clé anti-doublon site|commande)
%APPDATA%\SageBridge\journal.log	journal (archivé au-delà de 5 Mo dans journal.ancien.log)
%APPDATA%\SageBridge\installation.log	journal de l'installateur et des mises à jour
%TEMP%\SageBridge-maj\	installateurs téléchargés par la mise à jour automatique
HKCU\…\Uninstall\SageBridge	inscription dans Paramètres > Applications / Programmes et fonctionnalités
HKCU\…\CurrentVersion\Run	valeur SageBridge si « Démarrer avec Windows »

Code source (github.com/Gnaris/SageBridge)

Fichier	Rôle
SageBridge.cs	programme : normalisation, accès COM, mises à jour, historique, serveur HTTP, fenêtres
Setup.cs	installateur, mise à jour silencieuse, désinstalleur
app.manifest	exécution sans élévation, compatibilité Windows 10/11, haute résolution
build.cmd	compilation avec le csc.exe de Windows → bin\SageBridge-Setup.exe + .sha256
publier-version.cmd	création du dépôt, envoi du code et publication de la release
logo\, docs\	logo et icône ; cette documentation